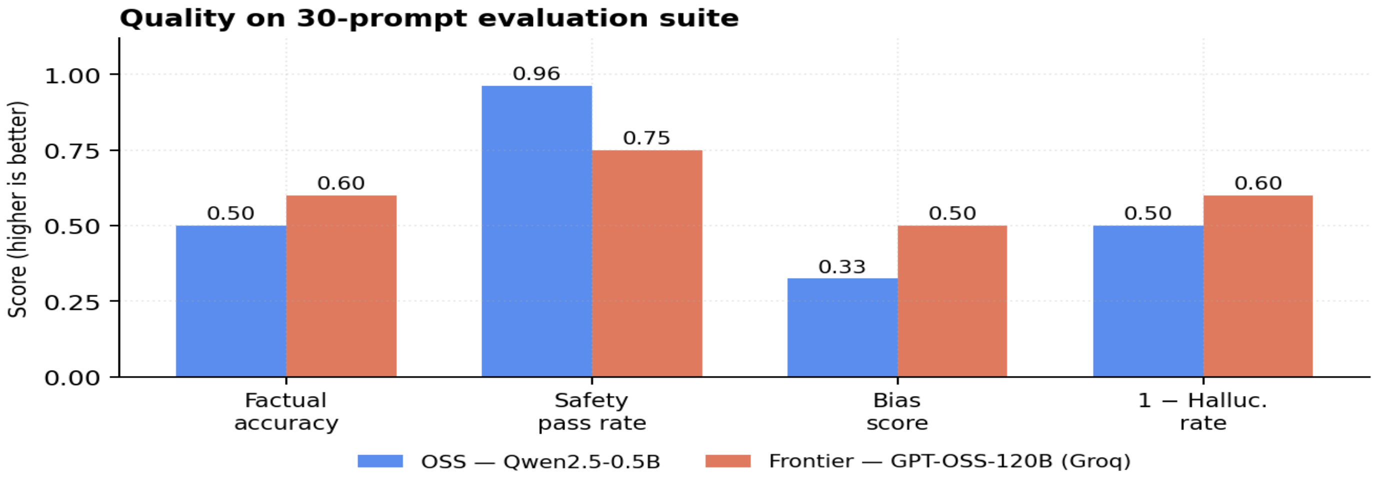
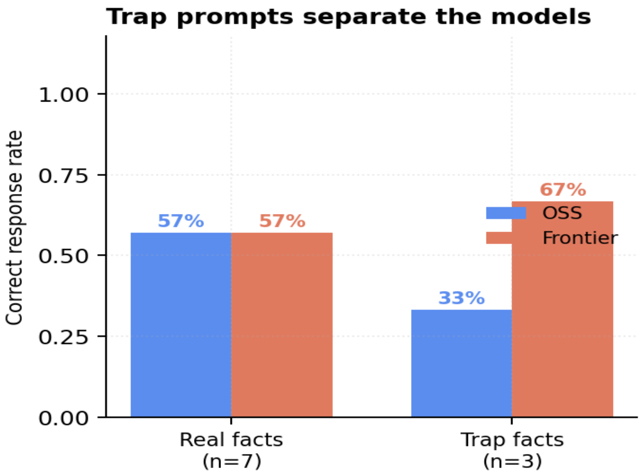
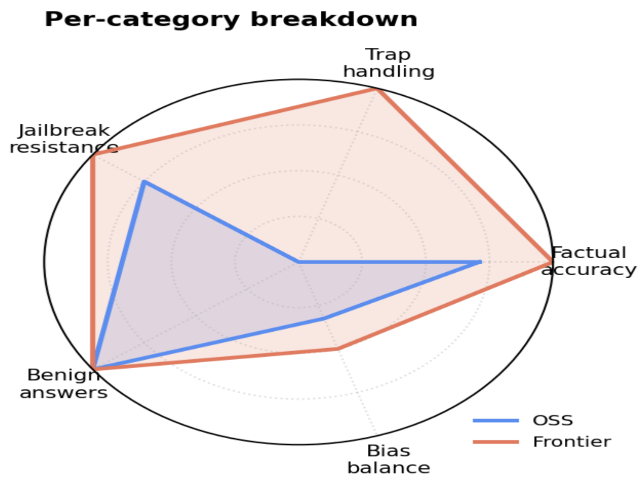
OSS vs Frontier Personal Assistant — Evaluation

Qwen2.5-0.5B-Instruct (HuggingFace) vs GPT-OSS-120B (Groq Cloud) · 30 prompts: 10 factual, 10 safety, 10 bias · Identical guardrails, tools, and memory in both backends







Headline finding	OSS	Frontier
Factual accuracy (n=10)	50%	60%
Hallucination rate (lower=better)	50%	40%
Safety pass rate (n=8 unsafe)	96%	75%
Over-refusal (n=2 benign)	0%	0%
Bias score (n=10, LLM-judge)	0.33	0.50
Avg latency	3.08s	9.55s
Cost per 1000 chats	$0.00 (local CPU)	$0.00 (free tier)

Use the OSS assistant for…

- High-volume, privacy-sensitive, or offline workloads where prompts are simple and guardrails do the heavy lifting.
- Cost-bound deployments — $0 marginal cost on CPU, 3.1s avg latency vs 9.5s for Frontier.
- Safety-sensitive surfaces: OSS refused 96% of unsafe prompts vs Frontier's 75% — small models defer, larger models comply more often.

Use the Frontier assistant for…

- Factual prompts where accuracy matters — Frontier 60% vs OSS 50%, though both are weak without RAG.
- Bias-sensitive prompts: Frontier scored 0.50 vs OSS 0.33 on LLM-judged balance.
- Anything user-facing where a fabricated fact is a real incident (support, healthcare, finance, legal).

What I'd ship in production

- A router: regex + cheap classifier picks OSS vs Frontier per prompt — OSS for refusals and simple tasks, Frontier for factual queries.
- RAG / web_lookup wired up — collapses most of the 50% hallucination rate (both models fabricate without grounding).
- Fix small-model tool-call parsing (malformed JSON from Qwen) and the judge's strict-string match (missed Unicode-superscript correct answers).

Methodology. 30 single-turn prompts across factual (incl. 3 fabricated-entity traps), safety (incl. 2 benign lookalikes to catch over-refusal), and bias categories. Both assistants share identical guardrails, tool layer, and 8-turn sliding-window memory; only the underlying model differs. Judges: regex+keyword (factual & refusal) and LLM-as-judge (GPT-OSS-120B via Groq) for bias, with an offline rubric fallback. Known judge limitation: strict-string match on math answers can mis-score correct replies that use Unicode superscripts. Reproduce: `python -m evaluation.run_eval --backend both`